

AWS Certified Generative AI Developer — Professional

The Use Case

- Healthcare organisation using FMs to analyse patient records, medical research and treatment outcomes for clinical decision support
- Strict **HIPAA** compliance while delivering useful insight to medical professionals

Task	Focus
3.1	Safety of the content — what is being said
3.2	Security of the data — who can see it, where it travels

3.1 blocked harmful prompts and hallucinations. 3.2 prevents **data leakage**.

1. Protected AI Environments

Network isolation — VPC endpoints and PrivateLink

Bedrock is a public AWS service, so by default traffic traverses the internet. In healthcare that is unacceptable.

Your VPC -> VPC Endpoint -> Bedrock
(stays inside the AWS network)

Endpoint type	Used for
Interface endpoint	Bedrock, Comprehend and most services — creates an ENI inside your VPC, and can be governed by security groups
Gateway endpoint	Only S3 and DynamoDB — a route table entry rather than an ENI

VPC architecture

Dedicated VPC for the healthcare platform
Private subnets for all AI processing components
NO direct internet access

- **Security groups** restrict traffic between components
- **Network ACLs** add a second network layer
- **Strict egress rules prevent data exfiltration** — if a component is compromised, it still cannot send data out

IAM for Bedrock

restrict to specific MODELS (only claude-3-sonnet)
restrict to specific FEATURES (InvokeModel yes, customization no)
restrict to specific OPERATIONS

- **Resource-based policies** apply to Knowledge Bases and other Bedrock resources
- **Condition keys** enforce encryption, source VPC, or required tags

Model	Basis for permission
RBAC	The user's role — doctor, nurse, administrator
ABAC	Attributes — department, clearance, patient assignment. Dynamic: new conditions don't require new policies.

AWS Lake Formation — fine-grained data governance

IAM controls access at bucket level. Lake Formation controls access **inside** a table.

Granularity	Example
Column-level	A doctor sees the diagnosis column but not the SSN column
Row-level	A doctor sees only their own patients' rows
Cell-level	Control down to an individual value

Data catalogs track and control access to AI-related datasets across the organisation. This is the same idea as metadata filtering in Task 1.4, but enforced by the service rather than by your query.

Monitoring and audit

Service	Role
CloudWatch	Alerts on suspicious data access patterns; custom metrics; anomaly detection
Logs Insights	Analyses access logs for security issues and policy violations
CloudTrail	Full audit of every Bedrock API call
GuardDuty	Threat detection for potential breaches
Security Hub	Centralised findings across all of the above

With **automated remediation workflows** for security events, and log retention aligned to healthcare compliance requirements.

2. Privacy-Preserving Systems

PII detection — two services, two places

Service	Operates on
Amazon Comprehend	Text, in real time — detects 25+ types of PII at request time
Amazon Macie	Data stored in S3 — automatically discovers and classifies sensitive data, with custom data identifiers for organisation-specific formats

"Does this request contain PII?"	-> Comprehend
"Where is PII sitting in my S3?"	-> Macie

Comprehend Medical is the healthcare variant: detects **PHI**, identifies medical entities in unstructured text, classifies sensitive medical information.

You need both — Comprehend gives inline protection, Macie tells you about sensitive data you didn't know was there.

Bedrock's native privacy guarantees

Customer data is not used to train or improve Bedrock's foundation models. This is the headline guarantee, and in healthcare it is a compliance requirement — data that enters model training becomes a permanent part of that model.

- Encryption **in transit and at rest** for all data processed by foundation models
- **Knowledge Bases** provide secure storage and retrieval of private data for grounding

Guardrails for privacy

PII filters	-> strip sensitive information from outputs
Custom topic filters	-> block specific categories
Applied to INPUTS and OUTPUTS	

Contextual controls are the interesting part: filtering adapts to **user role and access level**, with different guardrails for different clinical scenarios. The same model serves a doctor and a billing clerk with different guardrail configurations.

S3 Lifecycle — retention as a compliance control

transition to cold storage after a defined period
deletion policies aligned to retention requirements
versioning + lifecycle rules for audit purposes

Different retention per data category:

Patient records	-> 7 years (regulation)
Access logs	-> 3 years
Temp processing	-> 30 days

Same automatic-enforcement principle as DynamoDB TTL in Task 1.6 — nobody can forget. Transcript stresses "**compliant deletion processes with audit trails**": deleting isn't enough, you must be able to prove it happened.

Encryption and the processing pipeline

- KMS encryption at rest, **key rotation policies**, strict access controls on keys
- Pipeline: pre-processing Lambda sanitizes inputs → real-time PII detection and redaction → **log the PII detection events for compliance**

Logging what was redacted is a separate requirement from redacting it. A compliance audit asks not only "was PHI protected?" but "show me the record of each time it was detected."

3. Privacy-Focused AI Systems

PII has been detected — **now what?** Deleting it breaks context; keeping it breaks privacy. The techniques below keep the data useful while hiding identity.

Masking strategies

Technique	Example	Reversible
Redaction	"John Smith" → "[NAME]"	No
Tokenization	"John Smith" → "PATIENT_8472", real value held in a token mapping table	Yes
Pseudonymization	"John Smith" → "Patient A" — a realistic-looking but fake substitute, with a secure mapping table	Yes
Format-preserving encryption	4532-1234-5678-9010 → 8821-9945-2210-3387 — same length and shape , so legacy systems still accept it	Yes

Technique	Context retained	Choose when
Redaction	Least — the model can't tell two mentions are the same person	The sensitive value must simply be gone
Tokenization	Consistent references preserved	You need to recover the original later
Pseudonymization	Most — reads as natural text, so the model handles it normally	The model needs fluent, realistic input

Tokenization is the same technique used for cross-border data in Task 2.3.

Anonymization — dataset level rather than record level

Technique	How it works
Generalization	Age 34 → 30–40; ZIP 90210 → 902*; 2024-03-15 → March 2024. Less detail, utility retained.
Perturbation	Salary 85,000 → 84,200. The individual value changes; the aggregate stays close.
Synthetic data generation	Learn the statistical properties of the real dataset, then generate an entirely fake one. Safest — no real person is present — but hardest to get right.

Differential privacy

```
"How many cardiology patients?"  -> 500
one patient is discharged
"How many cardiology patients?"  -> 499
-> you have just identified that individual
```

Solution: add statistical noise to every answer.

```
true value: 500
returned:   502, or 498, or 501 (randomised)
```

Privacy budget: each query spends some privacy. Without a budget, asking the same question a hundred times averages the noise away and reveals the true value. When the budget is exhausted, no further queries are allowed.

The trade-off is direct: more noise means more privacy and less utility.

K-anonymity

```
Age 34, ZIP 90210, Cardiology  -> exactly one person
| generalize
Age 30-40, ZIP 902**, Cardiology -> 8 people    (k=5 satisfied)
```

Every record must look like at least $k-1$ others, so no individual record is uniquely identifiable.

De-identification pipeline

```
automated de-identification workflows
-> DIFFERENT techniques by data sensitivity
-> QUALITY CONTROL: is the data still useful?
```

The quality control step exists because **over-anonymising until the data is worthless is a common failure**. Privacy protection that destroys utility has solved nothing.

The full anonymization architecture

```
PHI Data
|
Anonymization Lambda
+-- Tokenization Service -> Token Mapping Table
+-- Format-Preserving Encryption
+-- Differential Privacy Engine
+-- De-identification Workflow
|
Data Quality Control
|
Anonymized Data Bucket
|
Bedrock <- Guardrails (content filter, response
                validation, contextual controls)
```

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.